Question 1

a) $MathStudents \cup CSStudents$ combines the two tables, any identical rows are ignored. Then we perform a natural join on that combined table with the Takes table where SID is the only matching column and is thus considered the only foreign key. So now we have:

SID	SName	Age	Course
1	Peter	40	Databases
2	Peter	20	Astronomy
2	Peter	20	Databases
1	Julia	30	Databases
2	Peter	40	Astronomy
2	Peter	40	Databases

Note that Anna is missing due to there not being $SID = 3$ row in Takes table to join with. After that we perform a query and select rows where $Age \geq 30\ OR\ SName \neq Peter$, and as a result we want to output these columns only: $SName\ AND\ Course$. Result:

SName	Course
Julia	Databases

Note that in this case, the restriction $Age \geq 30$ is redundant, but oh well...

b) Here we start with $\delta(CSStudents)$ which eliminates duplicates, but we don't have any, so nothing happens. Then we combine but with duplicates this time which does change the table slightly. We also do a natural join here too, but before that we make the output table smaller by stripping away the unnecessary columns (I see that we only want to count the number of rows, so that is fine). After all that we get this table:

SID	SName	Age
1	Peter	40
2	Peter	20
2	Peter	20
2	Peter	20
2	Peter	20
1	Julia	30
2	Peter	40
2	Peter	40

As you can see due to the bag union we got twice as many Peters of age 20, and Anna is missing due to the same reason as before. Lastly, we basically project CountSID and SName. We group by SName and count the number of rows having the same SID within those groups (CountSID). Age is ignored. The result thus becomes:

CountSID	SName
7	Peter
1	Julia

Question 2

a) The answer is a no. Here is an example of why: Suppose we got R(A) and S(A). Now let's evaluate the two where $\theta\ is\ R.A = S.A$ and $N\ is\ A \rightarrow B$. Equation on the left, $\rho_{A\rightarrow B}(R \bowtie_{R.A=S.A} S)$ performs an equi-join first and renames A to B. However, equation on the right, $\rho_{A\rightarrow B}(R) \bowtie_{R.A=S.A} \rho_{A\rightarrow B}(S)$, can no longer join R and S because neither has attribute A, both had A renamed to B so the output becomes NULL (I guess). To make the law hold we have to first rewrite the equi-join with the newly renamed attribute names. Since renaming might create clashes when joining we must also impose a restriction on N where when we split into two renamings ρ, we must also split N into two where each one must rename only the attributes of the one it is renaming. E.g. $\rho_{N_R}(R)$ and $\rho_{N_S}(S)$.

b) First of all, right anti-join is not idempotent. Suppose we have a table R and apply an anti-join on itself, $R \triangleleft R$. In the lecture "5 to 6" page 39 it is stated that anti-join is symmetric, which in our case means that $R \triangleleft R = R \triangleright R$. From that same page we resume with $R \triangleright R = R \setminus (R \ltimes R) = R \setminus \pi_{R_{attributes}}\ (R \bowtie R)$ and since in the lecture "5 to 6" page 43 it is explicitly stated that the natural join $\bowtie$ is idempotent, we resume the equation with $R \setminus \pi_{R_{attributes}}\ (R \bowtie R) = R \setminus \pi_{R_{attributes}}\ (R) = R \setminus R = \emptyset$. So, unless $R = \emptyset$, which is not the general case, $R \triangleleft R \neq R$.

It is also not commutative. Suppose we have tables R and S. For commutativity to hold $R \triangleleft S = S \triangleleft R$ must hold. Let's translate both of them as such: $R \triangleleft S = S \triangleright R = S \setminus \pi_{S_{attributes}}\ (S \bowtie R)$, and $S \triangleleft R = R \triangleright S = R \setminus \pi_{R_{attributes}}\ (R \bowtie S)$. Now, let Q be the result of $S \bowtie R$ (and $R \bowtie S$ too, because $\bowtie$ is commutative). If Q is not empty, then R and S relate some data. This data is being removed from both equations due to the difference operator which means that the two equations are not equal. This alone proves the non-commutativity of the right anti-join. As if that wasn't enough, if Q is empty, then R and S don't relate any data to begin with either. In fact, unless both R and S are equal to Ø, they always end up sharing no similar data whatsoever.

Lastly, right anti-join is not associative either. This time I will spare the details and just give a counterexample which disproves associativity. Suppose we have tables R, S, and Q where each one has one and the same attribute A. Now let's simply compute them both, $R \triangleright (S \triangleright Q)$ and $(R \triangleright S) \triangleright Q$. We get that $R \triangleright (S \triangleright Q) = R \triangleright \emptyset = R$ for the left side, and $(R \triangleright S) \triangleright Q = \emptyset \triangleright Q = \emptyset$ for the right side. Since $R \neq \emptyset$, the example is valid.

Question 3

I have approached this methodically and here is what I came up with and my reasons:

$R \bowtie S \equiv (R \bowtie S) \cup \pi_{R_{attributes}=NULL,\, S_{attributes}}(S \setminus \pi_{S_{attributes}}(S \bowtie R))$.

Firstly, the right outer join has everything that natural join has, so we add it first. Then we have to add whatever attributes that are missing. Those are the attributes of S that are not in the natural join, e.g. a right anti-join $R \triangleleft S \equiv S \setminus \pi_{S_{attributes}}(S \bowtie R)$. But we cannot use union on yet, because we are missing the NULLs of attributes of R, so we use the generalized projection to add those in: $\pi_{R_{attributes}=NULL,\, S_{attributes}}(R \triangleleft S)$. That's it.